

nano FP unit

Antonio Morán Muñoz

Universidad de Castilla-La Mancha

2026

Tabla de contenidos

Adaptador

Diferencia de exponentes

Adaptador

- ▶ El propósito de este módulo es adaptar los números de entrada a la codificación *binary32*, tal como se encuentra definida en el estándar IEEE-754, con independencia de la codificación inicial.

Adaptador

- ▶ El propósito de este módulo es adaptar los números de entrada a la codificación *binary32*, tal como se encuentra definida en el estándar IEEE-754, con independencia de la codificación inicial.
- ▶ La codificación de sistemas binarios se detalla en la Cláusula 3.4 del estándar IEEE-754, mientras que los parámetros de la codificación *binary32* se indica en la Tabla 3.5 de la Cláusula 3.6 de dicho estándar.

Adaptador

Formatos de representación binaria

Codificación binaria

Los formatos de representación binaria se codifican utilizando k bits, que se dividen en los siguientes campos:

Adaptador

Formatos de representación binaria

Codificación binaria

Los formatos de representación binaria se codifican utilizando k bits, que se dividen en los siguientes campos:

- ▶ S (1 bit): el signo.

Adaptador

Formatos de representación binaria

Codificación binaria

Los formatos de representación binaria se codifican utilizando k bits, que se dividen en los siguientes campos:

- ▶ S (1 bit): el signo.
- ▶ $E = e + bias$ (w bits): el exponente sesgado.

Adaptador

Formatos de representación binaria

Codificación binaria

Los formatos de representación binaria se codifican utilizando k bits, que se dividen en los siguientes campos:

- ▶ S (1 bit): el signo.
- ▶ $E = e + bias$ (w bits): el exponente sesgado.
- ▶ $T = d_1 d_2 \dots d_{p-1}$ ($t = p - 1$ bits): el significando.

Adaptador

Formatos de representación binaria

Codificación binaria

Los formatos de representación binaria se codifican utilizando k bits, que se dividen en los siguientes campos:

- ▶ S (1 bit): el signo.
- ▶ $E = e + bias$ (w bits): el exponente sesgado.
- ▶ $T = d_1 d_2 \dots d_{p-1}$ ($t = p - 1$ bits): el significando.

Nota

El bit d_0 está implícitamente codificado en el exponente sesgado.

Adaptador

Formato *binary32*

La siguiente tabla muestra el valor de los diferentes parámetros para el formato *binary32*.

<i>bias</i>	sign bit	<i>w</i>	<i>t</i>	<i>k</i>
127	1	8	23	32

Adaptador

Implementación

La implementación consiste en un bloque combinacional que llama a la función de conversión correspondiente, de acuerdo al formato de entrada.

Nota

De momento, sólo la conversión de *binary16* a *binary32* está implementada.

Adaptador

Implementación

```
always_comb begin : OUTPUT_LOGIC
    case (format_i)
        2'd1: begin
            num_o = b16_to_b32(num_i[15:0]);
        end
        default: begin
            num_o = num_i;
        end
    endcase
end
```

Tabla de contenidos

Adaptador

Diferencia de exponentes

Diferencia de exponentes

- ▶ El propósito de este módulo es calcular la diferencia entre los dos operandos para determinar cuál de los dos es menor.

Diferencia de exponentes

- ▶ El propósito de este módulo es calcular la diferencia entre los dos operandos para determinar cuál de los dos es menor.
- ▶ El operando menor deberá ser desplazado hacia la derecha.

Diferencia de exponentes

Implementación

Interfaz de entrada

exp_a_i : exponente del operando a

exp_b_i : exponente del operando b

Diferencia de exponentes

Implementación

Interfaz de entrada

exp_a_i : exponente del operando a

exp_b_i : exponente del operando b

Interfaz de salida

swap_o : flag de selección del operando

shift_o : número de bits de desplazamiento

Diferencia de exponentes

Implementación

- ▶ La lógica del módulo consiste en un bloque combinacional en el que se calcula la diferencia y se activan las señales correspondientes.

Diferencia de exponentes

Implementación

```
always_comb begin : OUTPUT_LOGIC
    swap_o = (exp_a_i >= exp_b_i) ? 0 : 1;

    if (swap_o) begin
        diff = exp_b_i - exp_a_i;
    end else begin
        diff = exp_a_i - exp_b_i;
    end

    if (diff > MAX_SHIFT) shift_o = MAX_SHIFT;
    else shift_o = diff;
end
```